Push_swap - Guia Completo para 42 School

📚 Conhecimentos Necessários

1. Estruturas de Dados

Stacks (Pilhas)

- LIFO (Last In, First Out)
- Operações: push, pop, peek
- Implementação com arrays ou linked lists

Linked Lists

- Singly linked lists
- Doubly linked lists (recomendado para push_swap)
- Operações: insert, delete, traverse

Arrays

- Indexação
- Ordenação
- Busca

2. Algoritmos de Ordenação

Conhecer (para comparação):

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- Merge Sort
- Radix Sort

Aplicar no Push_swap:

- Algoritmos adaptados para 2 stacks
- Chunking/Partitioning

- Indexação

3. Conceitos de Complexidade

Big O Notation:

- O(1) - Constante
- O(n) - Linear
- O(n^2) - Quadrática
- O(n log n) - Logarítmica

Para Push_swap:

- 3 números: ~3 operações
- 5 números: ~12 operações
- 100 números: <700 operações
- 500 números: <5500 operações

🎯 Melhores Algoritmos para Push_swap

Para 3 números (hardcoded)

Melhor abordagem: Casos específicos

```c
```

```
// Apenas 3 casos possíveis para 3 números
void sort_three(t_stack **a)
{
    int first = (*a)->value;
    int second = (*a)->next->value;
    int third = (*a)->next->next->value;

    if (first > second && second < third && first < third)
        sa(a);  // 2 1 3
    else if (first > second && second > third)
    {
        sa(a);
        rra(a); // 3 2 1
    }
    else if (first > second && second < third && first > third)
        ra(a);  // 3 1 2
    else if (first < second && second > third && first < third)
    {
        sa(a);
        ra(a);  // 2 3 1
    }
    else if (first < second && second > third && first > third)
        rra(a); // 1 3 2
}
```

Para 5 números

Estratégia: Push os 2 menores para B, ordena 3 em A, push back

1. Encontra os 2 menores números
2. Push para stack B (pb pb)
3. Ordena os 3 restantes em A
4. Push de volta para A (pa pa)

Operações: ~12 movimentos

Para 100 números - Chunk Algorithm (Recomendado)

Conceito: Divide números em chunks (grupos) e ordena progressivamente

Passos:

1. **Indexação:** Atribui índice a cada número (0 = menor, n-1 = maior)
2. **Push para B em chunks:**
   - Divide em 5 chunks (para 100 números)
   - Push números do chunk atual para B
   - Mantém os maiores no topo de B
3. **Push back para A ordenado:**
   - Encontra o maior em B
   - Rotate até topo
   - Push para A

Pseudocódigo:

```
chunk_size = stack_size / 5

for each chunk:
    while elements in this chunk exist in A:
        if top of A is in current chunk:
            pb
            if value < middle of chunk:
                rb
        else:
            ra

while B not empty:
    find max in B
    rotate max to top
    pa
```

Operações esperadas: 500-700 operações

Para 500 números - Optimized Chunk Algorithm

Diferença: Mais chunks, melhor optimização

Estratégia:

1. **Divide em 11 chunks** (aproximadamente)

2. **Push inteligente:**
    - Push para B mantendo ordem relativa
    - Usa rb para colocar menores no fundo
3. **Reconstrução optimizada:**
    - Calcula custo de cada movimento
    - Escolhe movimento mais barato

Operações esperadas: 4000-5500 operações

🏗️ Estrutura do Projeto

Arquitetura de Ficheiros

```
push_swap/
├── Makefile
├── includes/
│   └── push_swap.h
├── srcs/
│   ├── main.c
│   ├── parsing.c
│   ├── stack_init.c
│   ├── operations/
│   │   ├── push.c
│   │   ├── swap.c
│   │   ├── rotate.c
│   │   └── reverse_rotate.c
│   ├── algorithms/
│   │   ├── sort_small.c     // 2-5 números
│   │   ├── sort_chunk.c     // 100-500 números
│   │   └── cost_calculator.c
│   ├── utils/
│   │   ├── stack_utils.c
│   │   ├── error.c
│   │   └── free.c
│   └── indexing.c
└── libft/                   // Tua libft
```

💻 Estruturas de Dados

Stack Structure (Doubly Linked List)

```c
typedef struct s_stack
{
    int         value;      // Valor original
    int         index;      // Índice ordenado (0 = menor)
    int         pos;        // Posição atual na stack
    int         target_pos; // Posição alvo
    int         cost_a;     // Custo em stack A
    int         cost_b;     // Custo em stack B
    struct s_stack  *next;
    struct s_stack  *prev;
}   t_stack;
```

Program Structure

```c
typedef struct s_data
{
    t_stack *a;
    t_stack *b;
    int     size_a;
    int     size_b;
    int     total_size;
}   t_data;
```

🔧 Implementação das Operações

Push Operations

```c
```

```c
void	pa(t_data *data)
{
	t_stack *tmp;

	if (!data->b)
		return;
	tmp = data->b;
	data->b = data->b->next;
	if (data->b)
		data->b->prev = NULL;
	tmp->next = data->a;
	if (data->a)
		data->a->prev = tmp;
	data->a = tmp;
	write(1, "pa\n", 3);
}

void	pb(t_data *data)
{
	t_stack *tmp;

	if (!data->a)
		return;
	tmp = data->a;
	data->a = data->a->next;
	if (data->a)
		data->a->prev = NULL;
	tmp->next = data->b;
	if (data->b)
		data->b->prev = tmp;
	data->b = tmp;
	write(1, "pb\n", 3);
}
```

Swap Operations

```c
```

```c
void	sa(t_data *data)
{
	t_stack *first;
	t_stack *second;

	if (!data->a || !data->a->next)
		return;
	first = data->a;
	second = data->a->next;

	first->next = second->next;
	if (second->next)
		second->next->prev = first;
	second->prev = NULL;
	second->next = first;
	first->prev = second;
	data->a = second;
	write(1, "sa\n", 3);
}
```

Rotate Operations

```c
```

```
void	ra(t_data *data)
{
	t_stack *first;
	t_stack *last;

	if (!data->a || !data->a->next)
		return;
	first = data->a;
	last = data->a;

	while (last->next)
		last = last->next;

	data->a = first->next;
	data->a->prev = NULL;
	first->next = NULL;
	first->prev = last;
	last->next = first;
	write(1, "ra\n", 3);
}

void	rra(t_data *data)
{
	t_stack *last;
	t_stack *second_last;

	if (!data->a || !data->a->next)
		return;
	last = data->a;

	while (last->next)
		last = last->next;

	second_last = last->prev;
	second_last->next = NULL;
	last->prev = NULL;
	last->next = data->a;
	data->a->prev = last;
	data->a = last;
	write(1, "rra\n", 4);
}
```

🎓 Passo a Passo de Implementação

FASE 1: Setup Básico (Dia 1-2)

1. Parsing e Validação:

```c
// Verifica argumentos
// Converte strings para int
// Detecta duplicados
// Detecta overflow/underflow
// Retorna erro se inválido
```

2. Inicialização da Stack:

```c
// Cria linked list com valores
// Preenche stack A
// Stack B começa vazia
```

3. Operações básicas:

```c
// Implementa todas as 11 operações
// sa, sb, ss
// pa, pb
// ra, rb, rr
// rra, rrb, rrr
```

FASE 2: Algoritmos Simples (Dia 3-4)

1. Sort 2 números:

```c
if (a->value > a->next->value)
    sa(a);
```

2. Sort 3 números:

```c
// Hardcode dos 5 casos possíveis
// Máximo 2 operações
```

3. Sort 5 números:

```c
// Push 2 menores para B
// Sort 3 em A
// Push back para A
```

FASE 3: Indexação (Dia 5)

Criar índices ordenados:

```c
void	assign_index(t_stack *stack, int size)
{
	t_stack *current;
	t_stack *compare;
	int	index;

	current = stack;
	while (current)
	{
		index = 0;
		compare = stack;
		while (compare)
		{
			if (current->value > compare->value)
				index++;
			compare = compare->next;
		}
		current->index = index;
		current = current->next;
	}
}
```

FASE 4: Chunk Algorithm (Dia 6-8)

Implementação para 100 números:

```c
void    chunk_sort(t_data *data)
{
    int chunk_size;
    int chunk_num;
    int i;

    chunk_size = data->total_size / 5;
    chunk_num = 0;

    while (data->a)
    {
        i = 0;
        while (i < chunk_size && data->a)
        {
            if (data->a->index >= chunk_num * chunk_size &&
                data->a->index < (chunk_num + 1) * chunk_size)
            {
                pb(data);
                if (data->b->index < chunk_num * chunk_size + chunk_size / 2)
                    rb(data);
                i++;
            }
            else
                ra(data);
        }
        chunk_num++;
    }

    // Push back para A em ordem decrescente
    while (data->b)
    {
        move_max_to_top_b(data);
        pa(data);
    }
}
```

FASE 5: Optimização (Dia 9-10)

Cost Calculator:

```c
typedef struct s_cost
{
    int moves_a;
    int moves_b;
    int total;
}   t_cost;

t_cost  calculate_cost(t_stack *a, t_stack *b, int target)
{
    t_cost  cost;
    int     pos_a;
    int     pos_b;

    pos_a = get_position(a, target);
    pos_b = get_position(b, target);

    // Calcula custo de rotação em A
    if (pos_a <= stack_size(a) / 2)
        cost.moves_a = pos_a;
    else
        cost.moves_a = stack_size(a) - pos_a;

    // Calcula custo de rotação em B
    if (pos_b <= stack_size(b) / 2)
        cost.moves_b = pos_b;
    else
        cost.moves_b = stack_size(b) - pos_b;

    cost.total = cost.moves_a + cost.moves_b;
    return (cost);
}
```

Escolher movimento mais barato:

```c
```

```c
void	execute_cheapest_move(t_data *data)
{
	t_stack *current;
	t_cost  min_cost;
	t_cost  current_cost;
	int	target_index;

	current = data->b;
	min_cost.total = INT_MAX;

	while (current)
	{
		current_cost = calculate_cost(data->a, data->b, current->index);
		if (current_cost.total < min_cost.total)
		{
			min_cost = current_cost;
			target_index = current->index;
		}
		current = current->next;
	}

	execute_moves(data, min_cost, target_index);
}
```

📊 Benchmarks e Objetivos

Para 100%:

- **3 números:** ≤ 3 operações
- **5 números:** ≤ 12 operações
- **100 números:** < 700 operações
- **500 números:** < 5500 operações

Para 80%:

- **100 números:** < 1100 operações
- **500 números:** < 8500 operações

🧪 Testing

Testar com Checker

```bash
# Gerar números aleatórios
ARG=$(seq 1 100 | shuf | tr '\n' ' ')

# Contar operações
./push_swap $ARG | wc -l

# Verificar se está correto
./push_swap $ARG | ./checker_linux $ARG
# Deve retornar: OK
```

Script de Teste Automático

```bash
#!/bin/bash

count=0
total=0

for i in {1..100}
do
    ARG=$(seq 1 100 | shuf | tr '\n' ' ')
    ops=$(./push_swap $ARG | wc -l)
    total=$((total + ops))
    count=$((count + 1))

    result=$(./push_swap $ARG | ./checker_linux $ARG)
    if [ "$result" != "OK" ]; then
        echo "Error in test $i"
        exit 1
    fi
done

average=$((total / count))
echo "Average operations: $average"
```

🎯 Recursos Úteis

Visualizadores Online:

- **push-swap-visualizer** (GitHub)
- Ajuda a entender os movimentos visualmente

Testers:

- **push_swap_tester** (GitHub)
- Testa automaticamente com vários tamanhos

Algoritmos de Referência:

- **Turk Algorithm** (optimizado)
- **Radix Sort** (para muitos números)
- **Butterfly Algorithm** (alternativa)

💡 Dicas Importantes

DO:

✅ Começa pelo simples (2, 3, 5 números)
✅ Testa frequentemente
✅ Usa visualizador para debug
✅ Comenta o código
✅ Liberta toda a memória
✅ Trata todos os edge cases

DON'T:

❌ Não tentes optimizar desde o início
❌ Não copies código sem entender
❌ Não ignores memory leaks
❌ Não hardcodes valores mágicos
❌ Não te esqueças do Makefile

📝 Checklist Final

- [ ] Parsing completo e robusto
- [ ] Todas as 11 operações implementadas
- [ ] Sort para 2, 3, 5 números
- [ ] Algoritmo para 100 números
- [ ] Algoritmo para 500 números
- [ ] Sem memory leaks (valgrind)
- [ ] Error handling completo
- [ ] Makefile funcional (NAME, all, clean, fclean, re)
- [ ] Norminette OK
- [ ] Testes passam com checker
- [ ] Benchmarks atingidos

🚀 Timeline Sugerido

Semana 1:

- Dia 1-2: Setup, parsing, operações
- Dia 3-4: Sort 2, 3, 5
- Dia 5: Indexação

Semana 2:

- Dia 6-8: Chunk algorithm (100)
- Dia 9-10: Optimização (500)

Semana 3:

- Testing intensivo
- Optimizações finais
- Documentação

Boa sorte com o Push_swap! É um dos projetos mais desafiantes mas também dos mais rewarding! 💪